KUZLAB - KUZCODE - OPEN WEBUI

#AI #LocalLLM #Linux #OpenSource #kuzai.org #TheKuzNetwork

This guide assumes **KuzAI** is already installed to manipulate system parameters of different LLM models.
KuzAI README: http://github.com/Kusanagi8200/KuzAI

1 - SETUP HARDWARE

- Host: Nitro AN515-57 V1.14
- OS: Linux Mint 21.3 x86_64
- Kernel: 5.15.0-75-generic
- CPU: 11th Gen Intel i5-11400H (12 threads) @ 4.500GHz
- GPU (iGPU): Intel TigerLake-H GT1 [UHD Graphics]
- GPU (dGPU): NVIDIA GeForce RTX 3050 Mobile — 4G VRAM
- Memory: 32G — DDR4
- NVIDIA Driver: 575.64.03
- CUDA: 12.9

2 - INSTALLATION DRIVER NVIDIA

A - List detected GPUs

```
lspci | grep -i nvidia
```

Expected example:

```
01:00.0 VGA compatible controller: NVIDIA Corporation GA107M [GeForce RTX 3050] (rev a1)
```

B - Add the official NVIDIA PPA

```
add-apt-repository ppa:graphics-drivers/ppa
apt update
```

C - Identify the recommended driver

```
ubuntu-drivers devices
```

Expected example:

```
vendor   : NVIDIA Corporation
model    : GA107M [GeForce RTX 3050]
driver   : nvidia-driver-575 - distro non-free recommended
```

D - Install the recommended driver

```
apt install -y nvidia-driver-575
```

E - Post-installation checks

```
nvidia-smi
```

Expected example:

```
Sun Aug 31 17:01:25 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 575.64.03              Driver Version: 575.64.03      CUDA Version: 12.9     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 3050 ...    Off |   00000000:01:00.0 Off |                  N/A |
| N/A   47C    P8              7W /   60W |      18MiB /   4096MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            1421      G   /usr/lib/xorg/Xorg                        4MiB |
+-----------------------------------------------------------------------------------------+
```

3 - OLLAMA ENGINE INSTALLATION

```
curl -fsSL https://ollama.com/install.sh | sh
```

```
ollama --version
```

```
systemctl status ollama
```

Expected example:

```
● ollama.service - Ollama Service
     Loaded: loaded (/etc/systemd/system/ollama.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2025-08-31 14:03:23 CEST; 3h 5min ago
   Main PID: 4165 (ollama)
      Tasks: 19 (limit: 37882)
     Memory: 1.0G
        CPU: 12min 12.839s
     CGroup: /system.slice/ollama.service
             └─4165 /usr/local/bin/ollama serve

août 31 17:08:16 fhc ollama[4165]: [GIN] 2025/08/31 - 17:08:16 | 200 |      18.779µs |
127.0.0.1 | HEAD     "/"
```

4 - UPLOAD A MODEL WITH OLLAMA

A - Find a model and pull it

Browse models: https://ollama.com/search

```
ollama pull <MODEL:TAG>
```

B - Customize a model with KuzAI

KuzAI README: http://github.com/Kusanagi8200/KuzAI

```
ollama list
```

Expected example:

```
NAME                          ID              SIZE      MODIFIED
codellama:7b-instruct-q4_1    82177b511b42    4.2 GB    3 hours ago
kuzcode:latest                68cc9a57e5aa    4.2 GB    3 hours ago
kuzrag-full:latest            418d6e6bf328    4.1 GB    3 hours ago
```

```
ollama run kuzcode
```

Expected example:

```
>>> What's your purpose ?
KUZCode has been designed to assist users with their programming needs by providing accurate
information about the best practices of writing codes that can be used as a reference guide
when solving problems or creating new programs using AI technology.
```

5 - OPEN-WEBUI CHATBOT INSTALLATION

A - Create an open-webui service user

```
useradd --system --create-home --home-dir /var/lib/open-webui --shell /usr/sbin/nologin
openwebui
```

B - Application and data directory

```
mkdir -p /opt/open-webui
chown -R openwebui:openwebui /opt/open-webui

mkdir -p /var/lib/open-webui/data
chown -R openwebui:openwebui /var/lib/open-webui
```

C - Create the virtual environment and install the package

```
apt install -y python3.11 python3.11-venv
```

> Note: commands below are run as the `openwebui` user (no `sudo`).

```
runuser -u openwebui -- python3.11 -m venv /opt/open-webui/venv
runuser -u openwebui -- /opt/open-webui/venv/bin/pip install --upgrade pip
runuser -u openwebui -- /opt/open-webui/venv/bin/pip install open-webui
```

D - Create a systemd service

Create the unit file:

```
nano /etc/systemd/system/open-webui.service
```

Content:

```
[Unit]
Description=Open WebUI
After=network.target

[Service]
User=openwebui
Group=openwebui
WorkingDirectory=/var/lib/open-webui
Environment=DATA_DIR=/var/lib/open-webui/data
ExecStart=/opt/open-webui/venv/bin/open-webui serve --host 0.0.0.0 --port 8080
Restart=always
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full
ProtectHome=true

[Install]
WantedBy=multi-user.target
```

Enable + start:

```
systemctl daemon-reload
systemctl enable --now open-webui
systemctl status open-webui
```

Expected example:

```
● open-webui.service - Open WebUI
     Loaded: loaded (/etc/systemd/system/open-webui.service; disabled; vendor preset: enabled)
     Active: active (running) since Sun 2025-08-31 15:15:00 CEST; 2h 22min ago
   Main PID: 10267 (open-webui)
      Tasks: 47 (limit: 37882)
     Memory: 1.8G
        CPU: 44.357s
     CGroup: /system.slice/open-webui.service
             └─10267 /opt/open-webui/venv/bin/python3.11 /opt/open-webui/venv/bin/open-webui
serve --host 0.0.0.0 --port 8080
```

E - Bash aliases

```
cat >> /root/.bashrc <<'EOF'
alias owui-start='systemctl start open-webui'
alias owui-stop='systemctl stop open-webui'
alias owui-status='systemctl status open-webui'
EOF
. /root/.bashrc
```

6 - CONNECTION TO OPEN-WEBUI WEB CHATBOT

Open:

- http://localhost:8080
- http://127.0.0.1:8080
- http://<YOUR-IP>:8080

Create an admin user, then use the KuzGPT-like web interface.

Open-WebUI project: https://github.com/open-webui

WHY INSTALL A LOCAL OLLAMA + OPEN-WEBUI STACK ?

In a world where AI often means expensive, cloud-only solutions with little transparency, there is a real alternative: running your own models locally.

- **Ollama** provides a lightweight, GPU-optimized engine to run open-source models (LLaMA, Mistral, CodeLlama…) and your own custom creations.
- **Open-WebUI** provides a modern web interface to chat with models, manage prompts, and centralize workflows.

Benefits:

- Runs fully on your own hardware — no reliance on cloud providers.
- Works offline without internet access.
- Privacy: your data stays on your machine.
- Performance: GPU acceleration ensures faster responses.
- Flexibility: easily create and fine-tune models for your needs.
- Autonomy: no dependency on external services, zero recurring costs.

Combining Ollama and Open-WebUI provides a local, secure, and powerful stack for testing, development, or deploying AI agents without compromising between power and control.

OPEN WEBUI --- CUSTOM.CSS PERSISTENT INSTALLATION (SYSTEMD DROP-IN METHOD)

OBJECTIVE

Implement a persistent `custom.css` and background image (`bg.jpg`) for Open WebUI without modifying application source files and without risk of being overwritten during upgrades.

This method relies on a clean `systemd` drop-in configuration.

---

1. CREATE CUSTOM STATIC DIRECTORY

```
mkdir -p /var/lib/open-webui/custom-static
chown openwebui:openwebui /var/lib/open-webui/custom-static
```

---

2. PLACE YOUR FILES

Copy your files into the custom directory:

```
cp custom.css /var/lib/open-webui/custom-static/
cp bg.jpg     /var/lib/open-webui/custom-static/
chown openwebui:openwebui /var/lib/open-webui/custom-static/*
```

Verify:

```
ls -lh /var/lib/open-webui/custom-static
```

---

3. CREATE SYSTEMD DROP-IN CONFIGURATION

Create:

```
mkdir -p /etc/systemd/system/open-webui.service.d
nano /etc/systemd/system/open-webui.service.d/10-custom-static.conf
```

Insert:

```
[Service]
ExecStartPre=/usr/bin/install -D -m 0644 -o openwebui -g openwebui /var/lib/open-webui/custom-static/custom.css /opt/open-webui/venv/lib/python3.11/site-packages/open_webui/static/custom.css
ExecStartPre=/usr/bin/install -D -m 0644 -o openwebui -g openwebui /var/lib/open-webui/custom-static/bg.jpg /opt/open-webui/venv/lib/python3.11/site-packages/open_webui/static/bg.jpg
```

Save and exit.

---

4. RELOAD SYSTEMD AND RESTART

```
systemctl daemon-reload
systemctl restart open-webui
```

Verify service status:

```
systemctl status open-webui
```

---

5. VALIDATE STATIC FILES

Check from disk:

```
ls -lh /opt/open-webui/venv/lib/python3.11/site-packages/open_webui/static/custom.css
ls -lh /opt/open-webui/venv/lib/python3.11/site-packages/open_webui/static/bg.jpg
```

Check via HTTP:

```
curl -I http://127.0.0.1:8080/static/custom.css
curl -I http://127.0.0.1:8080/static/bg.jpg
```

Expected result:

- HTTP/1.1 200 OK
- Non-zero content-length

---

6. VERIFY PROCESS VIEW (OPTIONAL)

```
PID="$(systemctl show -p MainPID --value open-webui)"
STATIC_DIR="/opt/open-webui/venv/lib/python3.11/site-packages/open_webui/static"

ls -lh "/proc/$PID/root/$STATIC_DIR/custom.css"
ls -lh "/proc/$PID/root/$STATIC_DIR/bg.jpg"
```

Revision #3
Created 14 February 2026 18:42:44 by Kusanagi8200
Updated 15 February 2026 16:17:34 by Kusanagi8200